Code_Explanation_Concepts:

Code_1:

```
python

models.CheckConstraint(
  condition=models.Q(priority__lte=3),
  name="task_priority_max_3",
)
```

Here are the new terms in that code:

- **CheckConstraint** — a database rule that prevents a record from being stored if a specified condition is violated.
- **models.Q(...)** — a Django object used to express a database condition. It can also be used to build more complex queries.
- **priority__lte=3** — means that the priority must be less than or equal to 3. Here, lte stands for *less than or equal to*.
- **name="task_priority_max_3"** — gives the database constraint a unique, descriptive name.

Why do we need this when your serializer already validates priority?

Think of it as two layers of protection:

Layer	Responsibility
Serializer validation	Gives the API client a clear validation error before saving invalid data.
Database constraint	Prevents invalid data from being stored even when another code path bypasses the serializer.

Interview question: Why do you validate data in both the serializer and the database?

Model answer: Serializer validation provides user-friendly API errors, while database constraints enforce data integrity at the storage level. They complement each other because not every database write necessarily passes through the API serializer.

One important detail: we'll verify that the constraint is compatible with your database and existing records before applying the migration.

Code_2:

```
python

class Meta:
  ordering = ["-created_at"]


  indexes = [
    models.Index(fields=["created_by", "status"]),
  ]


  constraints = [
    models.CheckConstraint(
      condition=models.Q(
        priority__gte=0,
        priority__lte=3,
      ),
      name="task_priority_0_to_3",
    ),
  ]
```

Keep the rest of your model unchanged.

Step 2: Understand the new code

Let's break down the new terms.

1. What is a database constraint?

A database constraint is a rule enforced by the database itself. It helps prevent invalid data from being saved.

For example, your task priority should follow this rule:

Priority	Meaning	Allowed?
0	Lowest priority	Yes
1	Low priority	Yes
2	Medium priority	Yes
3	Highest priority	Yes
4	Outside the permitted range	No
-1	Negative priority	No

2. What is CheckConstraint?

models.CheckConstraint tells Django to create a database rule that checks whether a condition is true before allowing a record to be inserted or updated.

3. What is models.Q?

A Q object represents a query condition. Here, we're using it to express two requirements together:

python

priority__gte=0

priority__lte=3

- **gte** means *greater than or equal to*.
- **lte** means *less than or equal to*.

Both conditions must be satisfied.

4. Why give the constraint a name?

python

name="task_priority_0_to_3"

This provides a descriptive identifier for the database rule. It also helps Django manage the constraint through migrations.

Interview question

Why do we need a database constraint if the serializer already validates priority?

A good answer is:

“Serializer validation provides clear, user-friendly errors to API clients. A database constraint provides an additional layer of data integrity by enforcing the rule at the database level, even when a write bypasses the serializer”.

Code_3:

Currently, your serializer has this method:

```
python

def validate_priority(self, value):
  if value > 3:
    raise serializers.ValidationError(
      "Priority must be between 0 and 3."
    )


  return value
```

This is a good example of **field-level validation**.

Field-level validation means checking one specific field before the data is saved. In your case, **validate_priority()** checks the **priority** field.

We'll retain this method. Our next improvements will focus on two areas:

- **Consistent input validation:** Make sure invalid data is rejected with understandable error messages.
- **Automated validation tests:** Confirm that invalid status values and out-of-range priorities are rejected, while valid boundary values are accepted.

Code_4:

Open **tasks/tests.py** and add these methods inside your existing **TaskAPITests** class. Keep your current tests unchanged.

```
python

def test_reject_priority_above_maximum(self):
    payload = {
        "title": "Invalid priority task",
        "priority": 4,
    }

    response = self.client.post(
        "/api/tasks/",
        payload,
        format="json",
    )

    self.assertEqual(
        response.status_code,
        status.HTTP_400_BAD_REQUEST,
    )
    self.assertIn("priority", response.data)


def test_accept_maximum_priority(self):
    payload = {
        "title": "High priority task",
        "priority": 3,
    }

    response = self.client.post(
        "/api/tasks/",
        payload,
        format="json",
    )

    self.assertEqual(
        response.status_code,
        status.HTTP_201_CREATED,
    )


def test_reject_invalid_status(self):
    payload = {
        "title": "Invalid status task",
        "status": "invalid",
        "priority": 1,
    }
```

```
response = self.client.post(
    "/api/tasks/",
    payload,
    format="json",
  )

  self.assertEqual(
    response.status_code,
    status.HTTP_400_BAD_REQUEST,
  )
  self.assertIn("status", response.data)
```

Step 3: Understand the new testing concepts

- **self.client.post(...)** simulates an HTTP POST request to your API.
- **format="json"** tells Django REST Framework to send the request data as JSON.
- **HTTP_400_BAD_REQUEST** means the server rejected the submitted data as invalid.
- **HTTP_201_CREATED** means the server successfully created a resource.
- **self.assertIn("priority", response.data)** checks that the error response contains a field-specific error for priority.

The status test checks an important feature of your existing serializer: because **status** uses the model's defined choices, Django REST Framework should reject an unsupported value.

These tests are examples of **regression testing**: checking that expected behavior continues to work as the application evolves.

Interview tip: If asked why you validate priority in both the serializer and the database, explain that serializer validation gives API clients readable errors, while the database constraint protects stored data even when a write bypasses the serializer.

Once you've confirmed the tests still pass, we'll move on to a more significant improvement: making your API filtering and search robust, including handling invalid query parameters properly.

Step 1: Understand query parameters

A *query parameter* is a value provided in the URL after the **?** symbol.

For example:

```
text

/api/tasks/?status=todo&priority=2
```

Here:

- **status=todo** filters tasks by status.
- **priority=2** filters tasks by priority.
- **&** separates multiple parameters.

In Django REST Framework, **request.query_params** gives you access to these values.

Code_Below:

```
python


if request.method == "GET":
  tasks = Task.objects.filter(created_by=request.user)


  task_status = request.query_params.get("status")
```

```
priority = request.query_params.get("priority")
search = request.query_params.get("search")

# Validate status
valid_statuses = Task.Status.values

if task_status and task_status not in valid_statuses:
    return Response(
        {
            "status": [
                "Invalid status. Choose todo, in_progress, or done."
            ]
        },
        status=HTTP_400_BAD_REQUEST,
    )

# Validate priority
if priority is not None:
    try:
        priority_value = int(priority)
    except (TypeError, ValueError):
        return Response(
            {"priority": ["Priority must be an integer."]},
            status=HTTP_400_BAD_REQUEST,
        )

    if not 0 <= priority_value <= 3:
        return Response(
            {"priority": ["Priority must be between 0 and 3."]},
            status=HTTP_400_BAD_REQUEST,
        )

    tasks = tasks.filter(priority=priority_value)

if task_status:
    tasks = tasks.filter(status=task_status)

if search:
    tasks = tasks.filter(title__icontains=search.strip())

paginator = PageNumberPagination()
paginator.page_size = 5

page = paginator.paginate_queryset(tasks, request)
serializer = TaskSerializer(page, many=True)

return paginator.get_paginated_response(serializer.data)
```

Important: What changed?

1. task_status instead of status

We use a more descriptive local variable name. This avoids confusing the task's status with HTTP response status codes.

2. Task.Status.values

Your model's **Status** class defines the allowed values. This expression retrieves the valid stored values, so we don't need to hardcode the same list in our validation logic.

3. try and except

URL parameters arrive as strings. For example, **"2"** is a string, not an integer.

The **int()** function converts a numeric string to an integer. If someone sends **?priority=high**, conversion fails, and the except block returns a helpful **400 Bad Request** response instead of allowing a conversion error to become a server error.0

4. icontains

This performs a case-insensitive partial match on task titles. Searching for **django** can match titles containing **Django**, **DJANGO**, or **django**.

5. Why validate before filtering?

It makes the API's behavior predictable: invalid inputs receive clear error messages, while valid inputs are used to filter the user's own tasks.

Interview question

Why do we validate query parameters in an API?

A strong answer:

Query parameters are user input, so they should be validated before being used in database queries. Validation makes the API behavior predictable, provides meaningful client errors, and prevents invalid values from being interpreted as legitimate filter requests.

One design note: for a larger API, we could move filtering validation into a dedicated Django REST Framework serializer or filter class rather than keeping it in the view. For your current project, this is a reasonable incremental improvement.

---

2. Our next improvement: Better API architecture

So far, your filtering and validation work correctly. However, **task_list()** is beginning to handle several responsibilities:

- Processing HTTP requests.
- Validating query parameters.
- Filtering database records.
- Paginating results.
- Serializing data.
- Creating tasks.

This is manageable for a small project, but separating responsibilities makes larger APIs easier to maintain.

I recommend our next step be **moving query-parameter validation into a dedicated serializer**.

What is a query-parameter serializer?

You already use **TaskSerializer** to validate incoming task data. We can create another serializer specifically for query parameters, for example:

```
python

class TaskFilterSerializer(serializers.Serializer):
  status = serializers.ChoiceField(
    choices=Task.Status.choices,
    required=False,
  )
```

Here, **serializers.Serializer** defines validation rules without requiring a corresponding database model.

ChoiceField restricts the input to a predefined set of choices. **required=False** means clients can omit the parameter entirely.

We would extend this with priority and search validation, then let the view use the validated values. This keeps validation in one place and makes it easier to test independently.

Why this is valuable in interviews: you can explain separation of concerns—the principle of assigning different responsibilities to different components instead of putting everything into one function.

Manual validation vs serializer validation

Before

Validation in views.py

- Convert priority using **int()**.
- Catch conversion errors using **try/except**.
- Check whether priority is between 0 and 3.
- Check whether the status is valid.
- Return error responses manually.

Now

Validation in serializers.py

- **IntegerField** converts and validates integer input.
- **min_value=0** and **max_value=3** enforce the priority range.
- **ChoiceField** checks allowed status values.
- **CharField** validates search input and its length.
- DRF generates validation errors when the input is invalid.

Notice that we're not eliminating validation. We're **moving the validation rules into a component specifically designed for validation**.
